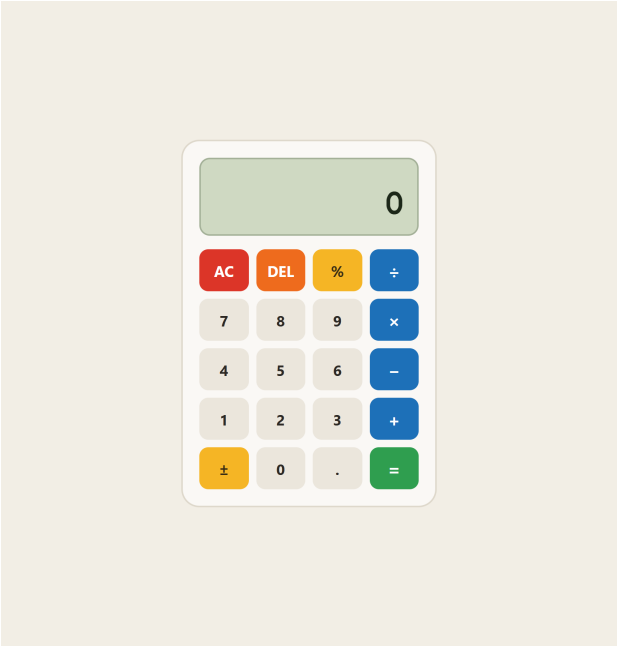
Web Development CSE343: Lab #3

Amir Atef Abdullah - 24P0310

Screenshots





Source Code Listings

page.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Retro Calculator</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="calculator-chassis">
        <section class="lcd-screen">
            <div id="previous-operand" class="lcd-history"></div>
            <div id="current-operand" class="lcd-value">0</div>
        </section>

        <main id="keypad" class="keypad">
            <button type="button" class="btn btn-red" data-action="clear">AC</button>
            <button type="button" class="btn btn-orange" data-action="delete">DEL</button>
            <button type="button" class="btn btn-yellow" data-action="percent">%</button>
            <button type="button" class="btn btn-blue" data-action="operator" data-value="÷">÷
                </button>

            <button type="button" class="btn btn-num" data-value="7">7</button>
            <button type="button" class="btn btn-num" data-value="8">8</button>
            <button type="button" class="btn btn-num" data-value="9">9</button>
            <button type="button" class="btn btn-blue" data-action="operator" data-value="×">×
                </button>

            <button type="button" class="btn btn-num" data-value="4">4</button>
            <button type="button" class="btn btn-num" data-value="5">5</button>
            <button type="button" class="btn btn-num" data-value="6">6</button>
            <button type="button" class="btn btn-blue" data-action="operator" data-value="−">−
                </button>

            <button type="button" class="btn btn-num" data-value="1">1</button>
            <button type="button" class="btn btn-num" data-value="2">2</button>
            <button type="button" class="btn btn-num" data-value="3">3</button>
            <button type="button" class="btn btn-blue" data-action="operator" data-value="+">+
                </button>

            <button type="button" class="btn btn-yellow" data-action="negate">±</button>
            <button type="button" class="btn btn-num" data-value="0">0</button>
            <button type="button" class="btn btn-num" data-action="decimal" data-value=".">.</
                button>
            <button type="button" class="btn btn-green" data-action="calculate">=</button>
        </main>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

style.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    background: #f2eee5;
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans
        -serif;
    color: #2c2824;
}

.calculator-chassis {
    width: 340px;
    background: #faf8f5;
    border: 2px solid #ded8cb;
    border-radius: 24px;
    padding: 22px;
}

.lcd-screen {
    background: #cfd9c2;
    border: 2px solid #a3b097;
    border-radius: 14px;
    padding: 14px 16px;
    margin-bottom: 18px;
    min-height: 104px;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    text-align: right;
    font-family: "Courier New", Courier, monospace;
}

.lcd-history {
    font-size: 15px;
    font-weight: bold;
    color: #5b6752;
    min-height: 20px;
    word-break: break-all;
}

.lcd-value {
    font-size: 48px;
    font-weight: bold;
    color: #1c2718;
    line-height: 1.1;
    word-break: break-all;
    transition: font-size 0.15s ease;
}

.keypad {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 10px;
}

.btn {
    height: 56px;
    border: none;
    outline: none;
    border-radius: 12px;
    font-family: inherit;
    font-size: 20px;
    font-weight: 700;
    cursor: pointer;
    user-select: none;
    display: flex;
    align-items: center;
```

```
    justify-content: center;
    transition: opacity 0.1s ease, filter 0.1s ease;
}

.btn:active {
    opacity: 0.85;
}

.btn-num {
    background: #ebe6dc;
    color: #2c2824;
}

.btn-num:hover {
    filter: brightness(0.97);
}

.btn-red {
    background: #dc3528;
    color: #ffffff;
}

.btn-red:hover {
    filter: brightness(0.95);
}

.btn-orange {
    background: #ee6b1d;
    color: #ffffff;
}

.btn-orange:hover {
    filter: brightness(0.95);
}

.btn-yellow {
    background: #f5b525;
    color: #382d15;
}

.btn-yellow:hover {
    filter: brightness(0.95);
}

.btn-blue {
    background: #1d70b8;
    color: #ffffff;
    font-size: 23px;
}

.btn-blue:hover {
    filter: brightness(0.95);
}

.btn-blue.active {
    background: #114775;
    color: #ffffff;
}

.btn-green {
    background: #2f9e4f;
    color: #ffffff;
    font-size: 24px;
}

.btn-green:hover {
    filter: brightness(0.95);
}
```

script.js

```
const previousOperandElement = document.getElementById('previous-operand');
const currentOperandElement = document.getElementById('current-operand');
const keypadElement = document.getElementById('keypad');
const operatorButtons = document.querySelectorAll('.btn-blue[data-action="operator"]');

let currentOperand = '0';
let previousOperand = '';
let currentOperator = null;
let shouldResetScreen = false;

function add(a, b) {
    return a + b;
}

function subtract(a, b) {
    return a - b;
}

function multiply(a, b) {
    return a * b;
}

function divide(a, b) {
    if (b === 0) {
        return null;
    }
    return a / b;
}

function operate(operator, a, b) {
    const numA = Number(a);
    const numB = Number(b);
    switch (operator) {
        case '+':
            return add(numA, numB);
        case '−':
        case '-':
            return subtract(numA, numB);
        case '×':
        case '*':
            return multiply(numA, numB);
        case '÷':
        case '/':
            return divide(numA, numB);
        default:
            return null;
    }
}

function roundResult(number) {
    return Math.round(number * 1000000000) / 1000000000;
}

function updateDisplay() {
    currentOperandElement.textContent = currentOperand;
    if (currentOperator !== null) {
        previousOperandElement.textContent = `${previousOperand} ${currentOperator}`;
    } else {
        previousOperandElement.textContent = previousOperand;
    }

    if (currentOperand.length > 9) {
        currentOperandElement.style.fontSize = '28px';
    } else if (currentOperand.length > 6) {
        currentOperandElement.style.fontSize = '38px';
    } else {
        currentOperandElement.style.fontSize = '48px';
    }
}

function clearOperatorHighlights() {
    operatorButtons.forEach((btn) => btn.classList.remove('active'));
}

```

```
function highlightOperator(operator) {
    clearOperatorHighlights();
    operatorButtons.forEach((btn) => {
        if (btn.dataset.value === operator) {
            btn.classList.add('active');
        }
    });
}

function appendNumber(number) {
    if (currentOperand === '0' || shouldResetScreen) {
        currentOperand = number;
        shouldResetScreen = false;
    } else {
        if (currentOperand.length >= 12) {
            return;
        }
        currentOperand += number;
    }
    clearOperatorHighlights();
    updateDisplay();
}

function appendDecimal() {
    if (shouldResetScreen) {
        currentOperand = '0.';
        shouldResetScreen = false;
        clearOperatorHighlights();
        updateDisplay();
        return;
    }
    if (!currentOperand.includes('.')) {
        currentOperand += '.';
    }
    clearOperatorHighlights();
    updateDisplay();
}

function toggleSign() {
    if (currentOperand === '0' || currentOperand === 'Error') {
        return;
    }
    if (currentOperand.startsWith('-')) {
        currentOperand = currentOperand.slice(1);
    } else {
        currentOperand = '-' + currentOperand;
    }
    updateDisplay();
}

function clearAll() {
    currentOperand = '0';
    previousOperand = '';
    currentOperator = null;
    shouldResetScreen = false;
    clearOperatorHighlights();
    updateDisplay();
}

function calculatePercent() {
    if (currentOperand === 'Error') {
        return;
    }
    const value = Number(currentOperand);
    currentOperand = roundResult(value / 100).toString();
    clearOperatorHighlights();
    updateDisplay();
}

function deleteDigit() {
    if (currentOperand === 'Error' || shouldResetScreen) {
        return;
    }
    if (currentOperand.length === 1 || (currentOperand.length === 2 && currentOperand.
        startsWith('-'))) {
        currentOperand = '0';
    } else {
```

```
        currentOperand = currentOperand.slice(0, -1);
    }
    updateDisplay();
}

function setOperator(operator) {
    if (currentOperand === 'Error') {
        return;
    }
    if (currentOperator !== null && !shouldResetScreen) {
        evaluate();
    }
    previousOperand = currentOperand;
    currentOperator = operator;
    shouldResetScreen = true;
    highlightOperator(operator);
    updateDisplay();
}

function evaluate() {
    if (currentOperator === null || shouldResetScreen) {
        return;
    }
    const result = operate(currentOperator, previousOperand, currentOperand);
    if (result === null) {
        currentOperand = 'Error';
        previousOperand = '';
        currentOperator = null;
        shouldResetScreen = true;
        clearOperatorHighlights();
        updateDisplay();
        return;
    }
    previousOperand = `${previousOperand} ${currentOperator} ${currentOperand} =`;
    currentOperand = roundResult(result).toString();
    currentOperator = null;
    shouldResetScreen = true;
    clearOperatorHighlights();
    updateDisplay();
}

keypadElement.addEventListener('click', (event) => {
    const button = event.target.closest('button');
    if (!button) {
        return;
    }

    const action = button.dataset.action;
    const value = button.dataset.value;

    if (!action) {
        appendNumber(value);
        return;
    }

    switch (action) {
        case 'clear':
            clearAll();
            break;
        case 'delete':
            deleteDigit();
            break;
        case 'percent':
            calculatePercent();
            break;
        case 'negate':
            toggleSign();
            break;
        case 'operator':
            setOperator(value);
            break;
        case 'decimal':
            appendDecimal();
            break;
        case 'calculate':
            evaluate();
            break;
```

```
    }
});

window.addEventListener('keydown', (event) => {
    const key = event.key;
    if (key >= '0' && key <= '9') {
        appendNumber(key);
    } else if (key === '.') {
        appendDecimal();
    } else if (key === '=' || key === 'Enter') {
        event.preventDefault();
        evaluate();
    } else if (key === 'Backspace') {
        deleteDigit();
    } else if (key === 'Escape' || key.toLowerCase() === 'c') {
        clearAll();
    } else if (key === '+') {
        setOperator('+');
    } else if (key === '-') {
        setOperator('−');
    } else if (key === '*') {
        setOperator('×');
    } else if (key === '/') {
        event.preventDefault();
        setOperator('÷');
    } else if (key === '%') {
        calculatePercent();
    }
});
```

repo link:
https://github.com/Auipp/Web-Dev-Labs-Repo